

Module : Kotlin (M202)

Devoir N° 1

Année de Formation 2024/2025

Filière : Développement Mobile Groupe : DEVM 101 - DEVM 102 Niveau : 2ème année

Exercice 1 :

Créer un système de gestion de bibliothèque utilisant les concepts de la programmation orientée objet en Kotlin. Ce système permettra de gérer des livres, des utilisateurs et les emprunts de livres.

→ Classe Personne (Classe de base) :

◆ Propriétés :

- nom : String
- prenom : String
- email : String

◆ Constructeur qui initialise ces propriétés.

◆ Méthode afficherInfos() qui affiche les informations de la personne.

→ Classe Utilisateur (Héritée de Personne) :

◆ Propriétés supplémentaires :

- idUtilisateur : Int
- emprunts : MutableList<Emprunt> (Liste des emprunts de l'utilisateur)

◆ Constructeur qui initialise les propriétés de la personne et l'ID utilisateur.

◆ Méthode emprunterLivre(livre: Livre, dateEmprunt: String) : permet à l'utilisateur d'emprunter un livre (ajoute un emprunt dans la liste des emprunts).

◆ Méthode afficherEmprunts() : affiche tous les emprunts de l'utilisateur.

→ Classe Livre :

◆ Propriétés :

- titre : String

- auteur : String
- isbn : String
- nombreExemplaires : Int
- ◆ Constructeur qui initialise ces propriétés.
- ◆ Méthode afficherDetails() : affiche les détails du livre.
- ◆ Méthode disponiblePourEmprunt() : retourne true si au moins un exemplaire est disponible, sinon false.
- ◆ Méthode mettreAJourStock(nouveauStock: Int) : met à jour le nombre d'exemplaires disponibles.

→ **Classe Emprunt :**

- ◆ Propriétés :
 - utilisateur : Utilisateur
 - livre : Livre
 - dateEmprunt : String
 - dateRetour : String? (nullable, car la date de retour peut ne pas être connue au moment de l'emprunt)
- ◆ Constructeur qui initialise ces propriétés.
- ◆ Méthode afficherDetails() : affiche les détails de l'emprunt.
- ◆ Méthode retournerLivre() : marque le livre comme retourné et met à jour la date de retour (met à jour aussi le stock du livre).

→ **Classe abstraite GestionBibliotheque :**

- ◆ Propriétés :
 - utilisateurs : MutableList<Utilisateur>
 - livres : MutableList<Livre>
- ◆ Méthodes :
 - ajouterUtilisateur(utilisateur: Utilisateur)
 - ajouterLivre(livre: Livre)
 - afficherTousLesLivres()
- ◆ Cette classe ne doit pas être instanciée directement.

→ Classe Bibliotheque (Hérite de GestionBibliotheque) :

- ◆ Implémente les méthodes de la classe abstraite.
- ◆ Ajoute une méthode rechercherLivreParTitre(titre: String) qui renvoie un livre dont le titre correspond.

Questions :

1. Créez plusieurs livres et utilisateurs.
2. Ajoutez les livres et les utilisateurs dans la bibliothèque.
3. Permettez à des utilisateurs d'emprunter des livres.
4. Affichez les informations des livres, des utilisateurs et des emprunts.
5. Faites retourner un livre par un utilisateur et mettez à jour les informations correspondantes.

Exercice 2 :

Créer un système de gestion d'un parc automobile en Kotlin. Ce système gère des véhicules de différents types, leurs conducteurs et leurs réservations. L'objectif est de mettre en pratique les principes de la programmation orientée objet, notamment l'héritage, l'encapsulation, le polymorphisme, les interfaces, les classes abstraites, et la gestion des exceptions.

Partie 1 : Classes de Base

→ Classe abstraite Vehicule :

- ◆ Propriétés :
 - immatriculation : String
 - marque : String
 - modele : String
 - kilometrage : Int
 - disponible : Boolean (défaut à true)
- ◆ Constructeur qui initialise ces propriétés.

◆ Méthodes :

- afficherDetails() : affiche les détails du véhicule.
- estDisponible() : retourne si le véhicule est disponible ou non.
- marquerIndisponible() : rend le véhicule indisponible.
- marquerDisponible() : rend le véhicule disponible.
- mettreAJourKilometrage(km: Int) : met à jour le kilométrage.

→ **Classe Voiture (hérite de Vehicule) :**

◆ Propriétés supplémentaires :

- nombrePortes : Int
- typeCarburant : String (par exemple, essence, diesel, électrique)

◆ Constructeur qui initialise ces propriétés.

◆ Méthode afficherDetails() : redéfinie pour afficher les détails spécifiques d'une voiture.

→ **Classe Moto (hérite de Vehicule) :**

◆ Propriétés supplémentaires :

- cylindree : Int (en cm³)

◆ Constructeur qui initialise ces propriétés.

◆ Méthode afficherDetails() : redéfinie pour afficher les détails spécifiques d'une moto.

Partie 2 : Gestion des Réservations

→ **Classe Conducteur :**

◆ Propriétés :

- nom : String
- prenom : String
- numeroPermis : String

◆ Constructeur qui initialise ces propriétés.

◆ Méthode afficherDetails() : affiche les détails du conducteur.

→ **Classe Reservation :**

◆ Propriétés :

- vehicule : Vehicule
- conducteur : Conducteur
- dateDebut : String
- dateFin : String
- kilometrageDebut : Int
- kilometrageFin : Int? (nullable, car non connu au début de la réservation)

◆ Constructeur qui initialise ces propriétés.

◆ Méthodes :

- cloturerReservation(kilometrageRetour: Int) : clôture la réservation et met à jour les informations du véhicule.
- afficherDetails() : affiche les détails de la réservation.

Partie 3 : Gestion du Parc

→ Classe ParcAutomobile :

◆ Propriétés :

- vehicules : MutableList<Vehicule>
- reservations : MutableList<Reservation>

◆ Méthodes :

- ajouterVehicule(vehicule: Vehicule) : ajoute un véhicule au parc.
- supprimerVehicule(immatriculation: String) : supprime un véhicule du parc.
- reserverVehicule(immatriculation: String, conducteur: Conducteur, dateDebut: String, dateFin: String) : permet de réserver un véhicule pour un conducteur.
- afficherVehiculesDisponibles() : affiche tous les véhicules disponibles.
- afficherReservations() : affiche toutes les réservations en cours.

Partie 4 : Gestion des Exceptions

→ Classe d'exception `VehiculeIndisponibleException` :

- ◆ Hérite de `Exception`.
- ◆ Le message d'erreur doit indiquer que le véhicule n'est pas disponible.

→ Classe d'exception `VehiculeNonTrouveException` :

- ◆ Hérite de `Exception`.
- ◆ Le message d'erreur doit indiquer que le véhicule avec l'immatriculation donnée n'a pas été trouvé.

Questions :

1. Créez un parc automobile avec plusieurs voitures et motos.
2. Ajoutez plusieurs conducteurs.
3. Effectuez des réservations pour ces conducteurs et mettez à jour l'état des véhicules (disponible ou non).
4. Gérez des exceptions si un véhicule est déjà réservé ou s'il n'existe pas dans le parc.
5. Affichez les véhicules disponibles et les détails des réservations.